T1 染布（dye）

同一批布必须使用同一种染料，因此关键是维护这一批允许的染料范围。如果当前范围为 $[L, R]$，加入第 i 块布后，共同允许的范围就是

$$[\max(L, l_i), \min(R, r_i)].$$

左端点不超过右端点时，仍能放在同一批；否则必须结束当前批次，从第 i 块布开始新的一批。

这样总把当前批次延长到不能再延长为止。为什么不需要提前分批？设从某个位置出发，最远能延长到 p。任何合法方案的第一批都不可能超过 p；把它的第一批延长到 p，只会从后面的批次中删去一些开头的布，不会让那些批次变得不合法，也不会增加批次数。因此总能选取第一批恰好结束在 p 的最优方案，剩下的布继续同样处理即可。

用第一块布初始化 L, R，批次数初始化为 1。随后依次求交；交集为空时，批次数加一，并用当前这块布的范围重新初始化。端点相等时仍有一种染料可选，不能误判为空。

每块布只处理一次，时间复杂度为 $O(n)$。输入可以边读边处理，额外空间复杂度为 $O(1)$。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;
int main(){
    freopen("dye.in","r",stdin);
    freopen("dye.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n,l,r,x,y,ans = 1;
    cin >> n >> l >> r;
    for(int i = 2; i <= n; i++){
        cin >> x >> y;
        if(max(l,x) <= min(r,y)){
            l = max(l,x);
            r = min(r,y);
        }else{
            ans++;
            l = x;
            r = y;
        }
    }
    cout << ans << endl;
    return 0;
}
```

部分分思路

条件	分值	做法与复杂度
$n \le 2000$，测试点 $1 \sim 8$	40 分	令 $f[i]$ 表示前 i 块布最少分成几批。枚举最后一批的开头 j，从 i 向前扫描并维护交集；交集非空时用 $f[j-1]+1$ 更新 $f[i]$。交集为空后继续向前也不会恢复，可结束这次枚举。时间 $O(n^2)$，空间 $O(n)$。

T2 工坊互助（workshop）

二分完工时间

任务应该交给谁不容易直接确定，可以先反过来问：**给定完工时间 T，哪些工人做不完自己的任务，其他人能否帮他们做完？**如果能在 T 分钟内完成，给更多时间也一定能完成，因此可以二分最早的可行时间。

先说明为什么可以让每名工人优先做自己的任务。假设工人 i 正在做一件外来任务，而他的一件本职任务交给了工人 j，就把这两件任务的执行者交换。工人 i 改做本职任务，用时减少；工人 j 原本已经接手外来任务，交换后这件任务仍是外来任务或变成本职任务，单件用时和准备时间都不会增加。重复交换后，不会有人一边帮别人，一边让别人帮自己。

这时，如果某人还没做满 $\min(a_i, T)$ 件自己的任务，就把相应任务从其他人手中交还给他，也不会超过 T。所以，判断是否可行时，可以先让工人 i 完成 $\min(a_i, T)$ 件自己的任务。

接下来分别统计还需要别人帮忙的任务数，以及其他人最多能接手的任务数。

- 如果 $a_i > T$，这名工人做满 T 分钟后，还剩 $a_i - T$ 件任务需要交给别人。
- 如果 $a_i \le T$，这名工人做完自己的任务后，还剩 $T - a_i$ 分钟。要接手别人的任务，需要先留出 c_i 分钟准备，剩余时间每 b_i 分钟才能完成一件。因此，他最多能接手 $\left\lfloor \dfrac{\max(0, T - a_i - c_i)}{b_i} \right\rfloor$ 件任务。

把前一类的缺口相加得到 need，把后一类的接手数量相加得到 help。**当且仅当 help 不小于 need 时，T 可行**。因为需要交出任务的人和可以帮忙的人属于不同的两类，任何待分配任务都能交给任意一名有余量的帮手，不再有其他限制。

让每个人都独自完成本职任务，一定可以在 $A = \max a_i$ 分钟内完成。因此，在 $[0, A]$ 内二分；中点可行就缩小右边界，否则将左边界移到中点之后，最终得到最小可行时间。

一次判定扫描 n 名工人，二分进行 $O(\log(A+1))$ 次，总时间复杂度为 $O(n \log(A+1))$，空间复杂度为 $O(n)$。虽然答案不超过 10^9，累加的任务缺口可以达到 2×10^{14}，相关变量需要用 long long。

参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 2e5 + 10;
int n;
long long a[N],b[N],c[N];

bool check(long long t){
    long long need = 0,help = 0;
    for(int i = 1; i <= n; i++){
        if(a[i] > t){
            need += a[i] - t;
        }else if(t > a[i] + c[i]){
            help += (t - a[i] - c[i]) / b[i];
        }
    }
    return help >= need;
}

int main(){
    freopen("workshop.in","r",stdin);
    freopen("workshop.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    long long l = 0,r = 0;
    for(int i = 1; i <= n; i++){
        cin >> a[i] >> b[i] >> c[i];
        r = max(r,a[i]);
    }
    while(l < r){
        long long mid = (l + r) / 2;
        if(check(mid)) r = mid;
        else l = mid + 1;
    }
    cout << l << '\n';
    return 0;
}
```

部分分

条件	保证分值	做法与复杂度
$n \le 2$	测试点 $1 \sim 4$，20 分	一人时答案为 a_1。两人时，设任务较多的一人有 A 件，另一人有 B 件；只需考虑前者交给后者 x 件。$x > 0$ 时，两人的用时分别为 $A - x$ 与 $B + c + bx$，一减一增，只需检查两者相等处附近的整数及不互助的方案。对两名工人是 $O(1)$ 时间、$O(1)$ 空间。
$n \le 1000$ 且 $a_i \le 1000$，或 $n = 1$	测试点 $1, 5 \sim 8$，25 分	每件任务至少需要 1 分钟，总任务数为 S 时，答案至少为 $\lceil S/n \rceil$。从这个下界开始逐个枚举 T，用正文的缺口与帮忙容量判定，遇到第一个可行时间即停止。一般时间复杂度 $O(nA)$，空间复杂度 $O(n)$，其中 $A = \max a_i$；$n = 1$ 时，下界就是答案，只需一次判定。

第一种做法中，设帮手的参数为 b, c，计算 $p = \max\left(0, \left\lfloor \dfrac{A - B - c}{b + 1} \right\rfloor\right)$，检查 $x = 0, p, p + 1$ 中满足 $0 \le x \le A$ 的候选。$x = 0$ 时不需要准备，应单独按 $\max(A, B)$ 计算。对应程序把这个方法推广为枚举一对工人互助，其余人各自完成本职任务；完整实现的时间复杂度为 $O(n^2)$，空间复杂度为 $O(n)$，在 $n \le 2$ 时得到最优答案。

两种算法都能保证测试点 1，这部分不能重复计分。按输入条件分别使用，可以覆盖测试点 $1 \sim 8$，共 40 分。

T3 灯带（lantern）

从最后一段确定转移

长度限制针对的是一整段连续同色灯。考虑一个合法前缀，把它末尾最长的同色段拿掉，剩下的前缀必然以另一种颜色结尾，或者为空。因此，可以按最后一段的颜色和长度分类计数。

前缀用了 i 盏红灯、j 盏蓝灯时，已经填到了位置 $p = i + j$。假设最后一段是 t 盏红灯，它占据位置 $p - t + 1$ 到 p，除了满足 $t \le i$、$t \le k$，还不能包含任何固定为蓝色的位置。

记 b_p 为前 p 个位置中最后一个固定蓝色的位置，没有则为 0。红色末段的起点必须在 b_p 后面，所以还要求 $t \le p - b_p$。因此，这一段的长度恰好可以取

$$1 \le t \le x, \qquad x = \min(i, k, p - b_p).$$

这也说明，只记录两色已经使用的数量还不够：接上一整段红灯时，需要知道前缀是否以蓝灯结尾。于是定义 $f_{i,j}$ 和 $g_{i,j}$，分别表示使用恰好 i 盏红灯、j 盏蓝灯，满足对应位置的固定颜色及连续长度限制，并且以红灯、蓝灯结尾的方案数。

拿掉末尾的 t 盏红灯后，剩下的正是 $g_{i-t,j}$ 所表示的方案，因此

$$f_{i,j} = \sum_{t=1}^{x} g_{i-t,j}.$$

同理，记 r_p 为前 p 个位置中最后一个固定红色的位置，没有则为 0。设 $y = \min(j, k, p - r_p)$，就有

$$g_{i,j} = \sum_{t=1}^{y} f_{i,j-t}.$$

每个非空方案都有唯一的末尾最长同色段，所以这些分类不重不漏；区间中没有固定异色、前缀本身合法，也保证了接段后的方案合法。

用前缀和合并连续转移

第一条式子连续求和的是同一列中的 $g_{i-x,j}$ 到 $g_{i-1,j}$；第二条连续求和的是同一行中的 $f_{i,j-y}$ 到 $f_{i,j-1}$。维护

$$S^{g}_{i,j} = \sum_{u=0}^{i} g_{u,j}, \qquad S^{f}_{i,j} = \sum_{v=0}^{j} f_{i,v},$$

便能通过两项前缀和相减，在 $O(1)$ 时间算出一个状态。$x = 0$ 或 $y = 0$ 时，对应方案数为 0；减法后加上模数再取模。

为了统一处理“第一段之前没有灯”的情况，令 $f_{0,0} = g_{0,0} = 1$，分别供第一段蓝灯、第一段红灯使用。这里只是为两个转移设置起点，非空方案仍按末段颜色分开计数，不会重复。

按 i 从小到大、同一行中 j 从小到大计算：$f_{i,j}$ 只依赖之前的行，$g_{i,j}$ 只依赖当前行之前的列。算完两个状态后更新它们的前缀和。答案为 $(f_{n,m} + g_{n,m}) \bmod (10^9 + 7)$。

预处理最后一个固定颜色的位置需要 $O(n + m)$ 时间，动态规划需要 $O(nm)$ 时间，总时间为 $O(nm + n + m)$。代码保留四张二维表以及字符串和位置数组，空间为 $O(nm + n + m)$；最大范围约有 4×10^6 个状态位置，四张整数表约占 62 MiB。

参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 2010;
const int P = 1000000007;
int n,m,k;
int f[N][N],g[N][N],sf[N][N],sg[N][N];
int r[N * 2],b[N * 2];
string s;

int main(){
    freopen("lantern.in","r",stdin);
    freopen("lantern.out","w",stdout);
    cin >> n >> m >> k >> s;
    s = " " + s;
    for(int p = 1; p <= n + m; p++){
        r[p] = r[p - 1];
        b[p] = b[p - 1];
        if(s[p] == 'R') r[p] = p;
        if(s[p] == 'B') b[p] = p;
    }
    f[0][0] = 1;
    g[0][0] = 1;
    for(int i = 0; i <= n; i++){
        for(int j = 0; j <= m; j++){
            int p = i + j;
            if(p > 0){
                int x = min(i,min(k,p - b[p]));
                if(x > 0){
                    f[i][j] = sg[i - 1][j];
                    if(i > x) f[i][j] = (f[i][j] - sg[i - x - 1][j] + P) % P;
                }
                int y = min(j,min(k,p - r[p]));
                if(y > 0){
                    g[i][j] = sf[i][j - 1];
                    if(j > y) g[i][j] = (g[i][j] - sf[i][j - y - 1] + P) % P;
                }
            }
            sf[i][j] = f[i][j];
            sg[i][j] = g[i][j];
            if(j > 0) sf[i][j] = (sf[i][j] + sf[i][j - 1]) % P;
            if(i > 0) sg[i][j] = (sg[i][j] + sg[i - 1][j]) % P;
        }
    }
    cout << (f[n][m] + g[n][m]) % P << '\n';
    return 0;
}
```

部分分

条件	分值	做法与复杂度
$n+m \le 22$ 或 $k=1$	保证测试点 $1 \sim 7$，共 35 分	从左到右搜索每个位置的颜色，记录已用两色数量及末尾连续长度，遇到数量超限、固定颜色冲突或连续长度超限就剪枝。一般时间为 $O(2^{n+m})$；$k=1$ 时，首色确定后只能交替，搜索至多沿两条路径继续，时间降为 $O(n+m)$。空间为 $O(n+m)$。
$k=1$	保证测试点 $5 \sim 7$，共 15 分	只能红蓝交替，分别检验从红色和蓝色开始的两种序列，核对两色总数和所有固定位置。时间、空间均为 $O(n+m)$。
$n,m \le 120$ 或 $k=1$	保证测试点 $1 \sim 12$，共 60 分	沿用正文的末段分类，直接枚举末段长度并累加对应前缀的方案数。一般时间为 $O(nmk+n+m)$，空间为 $O(nm+n+m)$；$k=1$ 时，每个状态只有一个候选长度，时间降为 $O(nm+n+m)$。

第一种搜索已经覆盖前两个点组，共 35 分；第二种做法是直接利用交替结构获得其中 15 分的另一条路径。第三种算法保证的 60 分包含前两种做法的全部保证点，不能重复相加。

T4 高台（plateau）

先固定公共边长 L。高度为 $a_{i,j}$ 的方格至少要补 $\max(0, L - a_{i,j})$ 块积木：原本够高就不用补，否则恰好补到 L 最省。一个正方形内各格费用之和，就是搭建这座展台的最少费用。两座展台没有公共方格，因此总费用直接相加。

接下来要选出费用最小的两个不重叠正方形。关键在于：**两个正方形如果在行的范围上相交、在列的范围上也相交，就一定有公共方格**。因此，合法的两块要么一块完全在另一块上方，要么一块完全在另一块左侧。沿着两块之间的一条水平或竖直格线，就能把它们分开。

这让我们不必枚举每一对位置：固定下面的一块，只需知道它上方最便宜的一块；固定右边的一块，只需知道它左侧最便宜的一块。

先计算每个位置的费用。将每格的补高费用记为 $c_{i,j} = \max(0, L - a_{i,j})$，用二维前缀和 $s_{i,j}$ 表示前 i 行、前 j 列的费用总和。右下角为 (i, j) 的正方形费用为

$$C(i,j) = s_{i,j} - s_{i-L,j} - s_{i,j-L} + s_{i-L,j-L}.$$

这里 $i, j \geq L$。边长改变时，每格费用也会改变，所以每次判定都要重新建立前缀和。

为了快速找到上方或左侧最便宜的方块，维护 row_i：完全位于前 i 行的正方形的最小费用。先枚举所有方块，记录底边恰好在第 i 行的最小费用，再从上到下取前缀最小值，就能得到 row_i。同样得到 col_j：完全位于前 j 列的正方形的最小费用。范围内放不下方块时，最小费用记为正无穷。

再次枚举右下角为 (i, j) 的方块。它从第 $i - L + 1$ 行开始，因此上方另一块的底边必须不超过第 $i - L$ 行；两块的最低总费用就是 $C(i, j) + row_{i-L}$。同理，左侧另一块必须完全位于前 $j - L$ 列，总费用为 $C(i, j) + col_{j-L}$。只要这两种费用中有一个不超过 B，边长 L 就可行。每对合法方块至少属于其中一种情况，且取区域内最小费用不会让它们相交，所以这个判定既不漏解，也不会选到重叠方块。

最后利用单调性寻找最大边长。若两个边长为 L 的方块可行，分别在它们内部取一个边长为 $L - 1$ 的方块，仍不会重叠；需要补高的格子变少、每格要求的高度也降低，总费用不会增加。因此可以二分边长。

除了 $L \leq n, m$，两块还必须能沿某个方向并排放下，即 $2L \leq n$ 或 $2L \leq m$。所以二分上界取

$$q = \min\left(n, m, \left\lfloor \frac{\max(n, m)}{2} \right\rfloor\right).$$

答案初始为 0，覆盖放不下两块或预算不足的情况。一次判定用 $O(nm)$ 时间，总时间复杂度为 $O(nm \log(q + 1))$，空间复杂度为 $O(nm)$。最多处理 4×10^6 个方格，预算和费用前缀和都可能超过 32 位整数范围，统一用 long long 保存。

参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 2010;
const long long INF = 400000000000000000ll;
int n,m,a[N][N];
long long b,s[N][N],row[N],col[N];

bool check(int len){
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            int cost = max(0,len - a[i][j]);
            s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + cost;
        }
    }
    for(int i = 0; i <= n; i++) row[i] = INF;
    for(int j = 0; j <= m; j++) col[j] = INF;
    for(int i = len; i <= n; i++){
        for(int j = len; j <= m; j++){
            long long cost = s[i][j] - s[i - len][j] - s[i][j - len] + s[i - len][j - len];
            row[i] = min(row[i],cost);
            col[j] = min(col[j],cost);
        }
    }
    for(int i = 1; i <= n; i++) row[i] = min(row[i],row[i - 1]);
    for(int j = 1; j <= m; j++) col[j] = min(col[j],col[j - 1]);
    for(int i = len; i <= n; i++){
        for(int j = len; j <= m; j++){
            long long cost = s[i][j] - s[i - len][j] - s[i][j - len] + s[i - len][j - len];
            if(row[i - len] + cost <= b) return true;
            if(col[j - len] + cost <= b) return true;
        }
    }
    return false;
}

int main(){
    freopen("plateau.in","r",stdin);
    freopen("plateau.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m >> b;
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++) cin >> a[i][j];
    }
    int l = 1,r = min(min(n,m),max(n,m) / 2),ans = 0;
    while(l <= r){
        int mid = (l + r) / 2;
        if(check(mid)){
            ans = mid;
            l = mid + 1;
        }else{
            r = mid - 1;
        }
    }
    cout << ans << '\n';
    return 0;
}
```

部分分

下表的分值是对应算法保证通过的测试点得分，q 为正文给出的边长上界。

条件	分值	做法与复杂度
$n, m \le 8$	15 分（测试点 $1 \sim 3$）	枚举边长，逐格算出每个位置的方块费用，再枚举两个位置，检查行范围或列范围是否分离，并比较费用和。时间 $O(nmq^3 + n^2m^2q)$，空间 $O(nm)$。
所有格子的高度同为 h	15 分（测试点 $4 \sim 6$）	每个合法摆放的费用都相同，两块共需 $2L^2 \max(0, L-h)$ 块积木。枚举 $1 \sim q$ 的边长并检查预算。计入读入，时间为 $O(nm+q)$；所给程序存储矩阵，空间 $O(nm)$。
$n, m \le 250$ 或 $\min(n, m) \le 20$	35 分（测试点 $1 \sim 3$、$7 \sim 10$）	依次枚举边长，按正文的方法利用二维前缀和、行列前缀最小值检查能否选出两块，在首个不可行边长处停止。时间 $O(nmq)$，空间 $O(nm)$；该条件自然包含第一档。

第三种算法的 35 分已经包含第一档的 15 分。先判断是否全部等高，再使用第二种或第三种算法，可保证通过测试点 $1 \sim 10$，合计 50 分。